

Обробка та аналіз мережного трафіка

Кувавіна Софія

1. В завданнях 2-5 потрібно записати за допомогою *Wireshark* (також можливо використовувати *tshark*, *tcpdump* або *netsniff-ng*) весь трафік, що передається при їх виконанні, та зберегти його у файл (або в декілька файлів - як вам буде зручно). В кінці роботи потрібно буде провести узагальнення трафіка, що передавався, проте об'єднувати все в один файл не потрібно.

Я створила дві віртуальні машини Kali Linux, налаштувала мережу Host-only (VMnet1)

Host a (kali1)

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default
   link/ether 00:0c:29:5a:2d:7e brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:5a:2d:88 brd ff:ff:ff:ff:ff:ff
   inet 192.168.114.131/24 brd 192.168.114.255 scope global dynamic noprefixroute eth1
       valid_lft 1742sec preferred_lft 1742sec
   inet6 fe80::749b:4b7f:c223:8f9f/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Host b (kali2)

```
(kali@kali2)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
   link/ether 00:0c:29:f0:90:92 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:f0:90:9c brd ff:ff:ff:ff:ff:ff
   inet 192.168.114.133/24 brd 192.168.114.255 scope global dynamic noprefixroute eth1
       valid_lft 1731sec preferred_lft 1731sec
   inet6 fe80::3f43:7a3a:ac21:f60b/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Перевірка зв'язку (ping)

Host a (kali1)

```
(kali㉿kali)-[~]
$ ping 192.168.114.133
PING 192.168.114.133 (192.168.114.133) 56(84) bytes of data.
64 bytes from 192.168.114.133: icmp_seq=1 ttl=64 time=2.71 ms
64 bytes from 192.168.114.133: icmp_seq=2 ttl=64 time=1.47 ms
64 bytes from 192.168.114.133: icmp_seq=3 ttl=64 time=1.21 ms
64 bytes from 192.168.114.133: icmp_seq=4 ttl=64 time=2.79 ms
64 bytes from 192.168.114.133: icmp_seq=5 ttl=64 time=0.743 ms
^C
— 192.168.114.133 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 0.743/1.785/2.793/0.824 ms
```

Host b (kali2)

```
(kali㉿kali2)-[~]
$ ping 192.168.114.131
PING 192.168.114.131 (192.168.114.131) 56(84) bytes of data.
64 bytes from 192.168.114.131: icmp_seq=1 ttl=64 time=5.32 ms
64 bytes from 192.168.114.131: icmp_seq=2 ttl=64 time=0.858 ms
64 bytes from 192.168.114.131: icmp_seq=3 ttl=64 time=0.500 ms
64 bytes from 192.168.114.131: icmp_seq=4 ttl=64 time=1.21 ms
64 bytes from 192.168.114.131: icmp_seq=5 ttl=64 time=1.38 ms
^C
— 192.168.114.131 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4009ms
rtt min/avg/max/mdev = 0.500/1.854/5.321/1.759 ms
```

2. *Перевірте взаємну досяжність між hostA і hostB та, послуговуючись tcpdump, запишіть окремими .pcap файлами пакети ping-request (три послідовних пакети у одному файлі), ARP-request та ARP-reply для hostA та hostB. *Для скидання кешу ARP використовуйте команду arp -d (або arp -d для Windows).*

Ping

Далі я використала tcpdump для перехоплення ICMP пакетів на eth1 і зробила фільтр, щоб залишити тільки ping-запити

```
(kali㉿kali)-[~]
$ sudo tcpdump -i eth1 icmp and icmp[0]=8 -c 3 -w ping_request.pcap
[sudo] password for kali:
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
3 packets captured
3 packets received by filter
0 packets dropped by kernel
```

У результаті я отримала 3 ICMP Echo Request пакети та зберегла їх у файл ping_request.pcap

ARP request

Перед початком я скинула ARP-кеш, після цього я запустила tcpdump для перехоплення ARP-запитів, використовуючи фільтр, де значення opcode = 1

```
(kali@kali)-[~]
$ sudo arp -d 192.168.114.133

(kali@kali)-[~]
$ sudo tcpdump -i eth1 arp and arp[6:2]=1 -c 1 -w arp_request.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 2621
44 bytes

1 packet captured
1 packet received by filter
0 packets dropped by kernel
```

Після генерації трафіку я отримала 1 ARP request пакет

ARP reply

Далі я виконала перехоплення ARP-відповіді, змінивши значення фільтра на opcode = 2

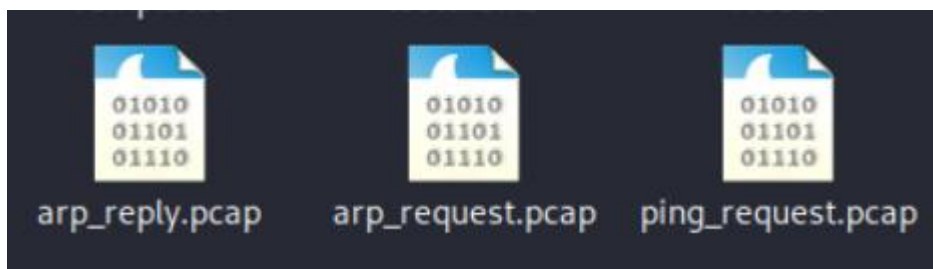
```
(kali@kali)-[~]
$ sudo arp -d 192.168.114.133

(kali@kali)-[~]
$ sudo tcpdump -i eth1 arp and arp[6:2]=2 -c 1 -w arp_reply.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 2621
44 bytes

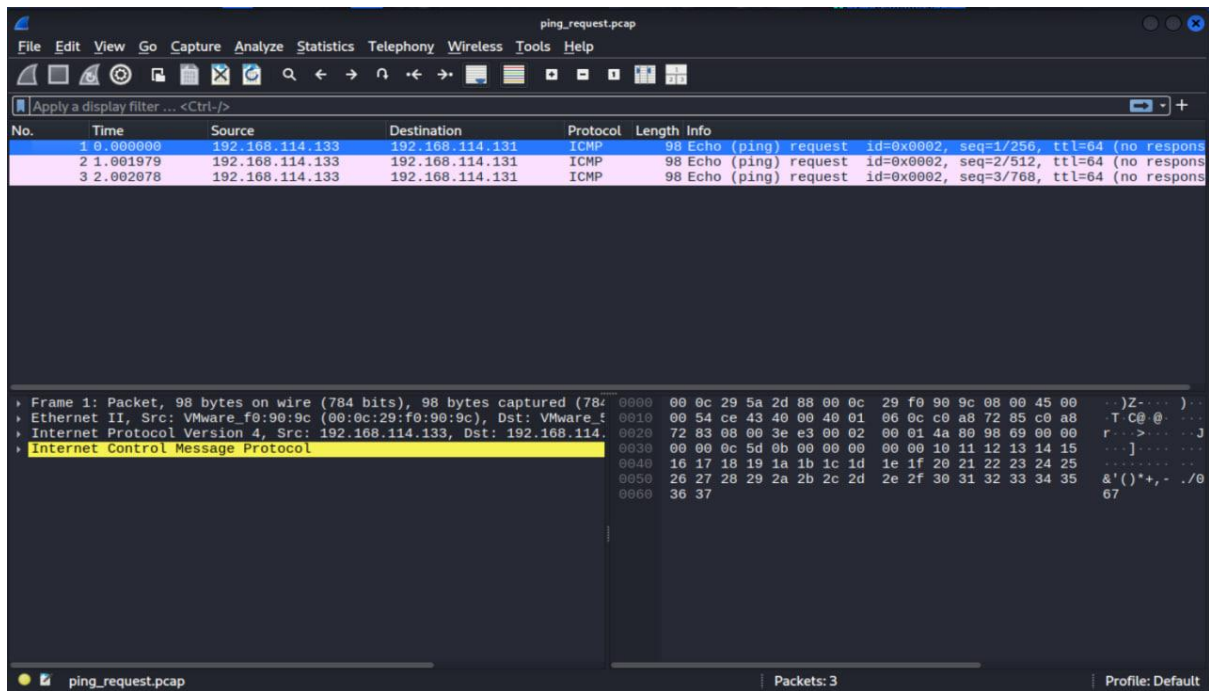
1 packet captured
1 packet received by filter
0 packets dropped by kernel
```

У результаті я отримала 1 ARP reply пакет

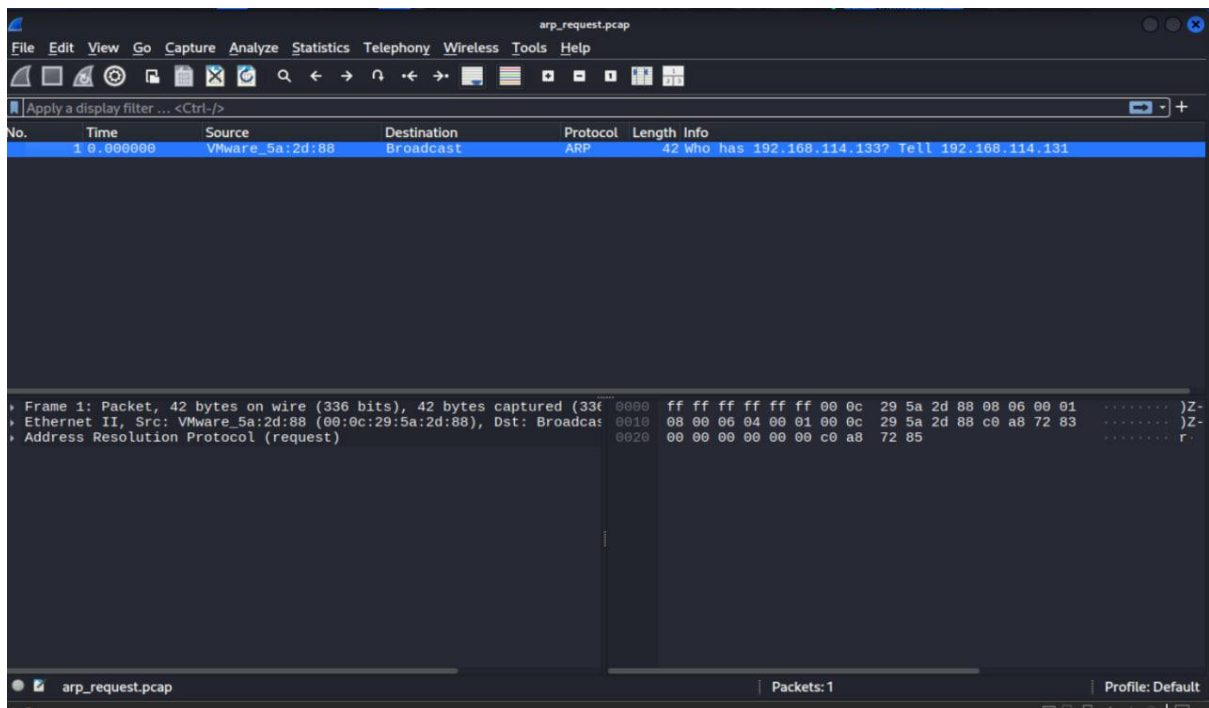
В підсумку маємо три файли



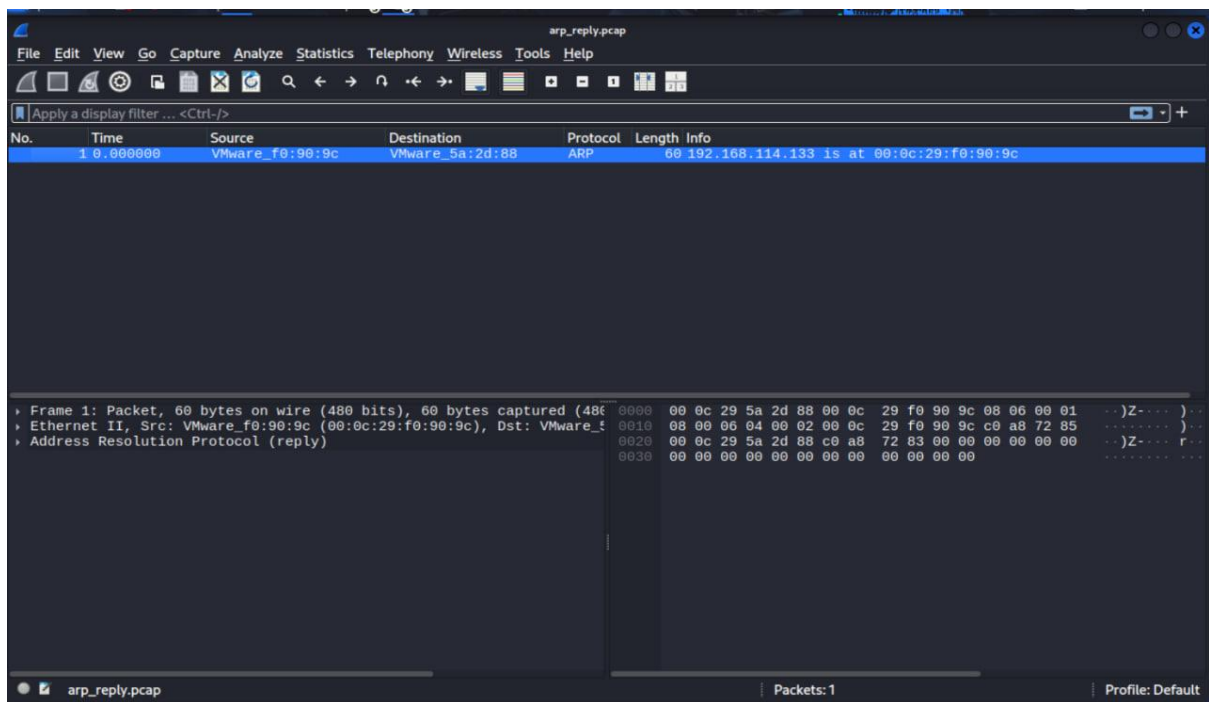
Після відкриття у Wireshark видно ICMP Echo Request (Type 8), в Source/Destination відображаються IP hostA та hostB відповідно до напрямку ping



Для ARP request - opcode = 1 (request), пакет надсилається broadcast, в Info є запит Who has IP hostB? Tell IP hostA



Для ARP request - opcode = 2 (reply), відповідь йде unicast, hostB повертає свою MAC-адресу для IP.

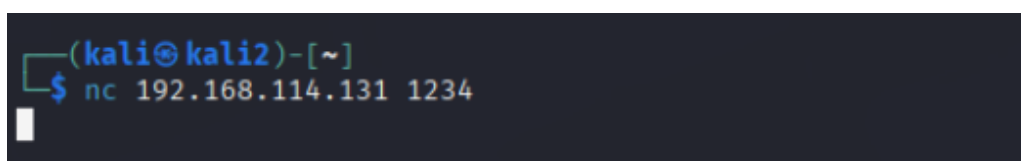


3. *Запустіть, використовуючи netcat, між hostA (server) та hostB (client) з'єднання: nc -l //на сервері та nc //на клієнті після чого наберіть на стороні клієнта декілька строк та завершіть з'єднання а потім передайте деякий файл: nc -l -p >filename //на сервері та cat filename | nc -q 0 //на клієнті За допомогою утиліт netstat (та TCPView для Windows, можна знайти на сайтах Microsoft Sysinternals) з'ясуйте, які мережні сервіси працюють на кожному хості. Які порти вони використовують? Які програми виконують дані сервіси? *команди netstat -atun для Linux та netstat -an для Windows З'ясуйте, скільки сеансів TCP було відкрито між цими двома хостами. Які в цих сеансах передавались файли. Які коди повідомлень та які заголовки використовувались.*

Запускаю сервер на *hostA* (kali1)



Клієнт *hostB* (kali2) підключаюся



На клієнті набрала кілька рядків

```
(kali㉿kali2)-[~]  
$ nc 192.168.114.131 1234  
hello bkm!  
lalalala  
█
```

Бачу на сервері ті ж рядки, з'єднання успішне

```
(kali㉿kali)-[~]  
$ nc -l -p 1234  
hello bkm!  
lalalala  
█
```

```
nc -l -p <port> >filename //на сервері та  
cat filename | nc -q 0 <ip> <port> //на клієнті
```

На сервері я запустила прослуховування порту та запис даних у файл

```
(kali㉿kali)-[~/lab1bkm]  
$ nc -l -p 1235 > lab.txt
```

На клієнті я створила файл

```
(kali㉿kali2)-[~/lab1bkm]  
$ echo "i like potato and dont like tomato" > lab.txt  
  
(kali㉿kali2)-[~/lab1bkm]  
$ cat lab.txt  
i like potato and dont like tomato
```

Далі я передала файл з клієнта на сервер за допомогою команди

```
(kali㉿kali2)-[~/lab1bkm]  
$ cat lab.txt | nc -q 0 192.168.114.131 1235
```

У результаті файл успішно відправлено на сервер, переглядаю за допомогою команди cat

```
(kali@kali)-[~/lab1bkm]
$ cat lab.txt
i like potato and dont like tomato
```

*За допомогою утиліт netstat (та TCPView для Windows, можна знайти на сайтах Microsoft Sysinternals) з'ясуйте, які мережні сервіси працюють на кожному хості. Які порти вони використовують? Які програми виконують дані сервіси? *команди netstat -atun для Linux та netstat -an для Windows*

За допомогою netstat -atun я подивилась які сервіси на хостах слухають порти

Бачу з'єднання з клієнтом

```
(kali@kali)-[~/lab1bkm]
$ netstat -atun
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:8888             0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:139              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:445              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:443              0.0.0.0:*               LISTEN
tcp6       0      0 :::8888                  :::*                    LISTEN
tcp6       0      0 :::139                   :::*                    LISTEN
tcp6       0      0 :::445                   :::*                    LISTEN
udp        0      0 192.168.114.131:68      192.168.114.254:67     ESTABLISHED
```

На клієнті бачу з'єднання з сервером

```
(kali@kali2)-[~/lab1bkm]
$ netstat -atun
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:8888             0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:139              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:443              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:445              0.0.0.0:*               LISTEN
tcp6       0      0 :::8888                  :::*                    LISTEN
tcp6       0      0 :::139                   :::*                    LISTEN
tcp6       0      0 :::445                   :::*                    LISTEN
udp        0      0 192.168.114.133:68      192.168.114.254:67     ESTABLISHED
```

На обох машинах видно, що відкриті порти 8888, 139, 445, 443, і ці порти знаходяться у стані LISTEN, отже, можна зробити висновок, що на обох хостах працюють стандартні мережеві служби Linux.

Порти 139 та 445 використовуються для мережних служб обміну файлами, порт 443 для захищених з'єднань, а порт 8888 використовується системними або користувацькими програмами, для виконання попереднього завдання я використовувала програму netcat (nc), яка відкривала порти для передачі даних між хостами.

З'ясуйте, скільки сеансів TCP було відкрито між цими двома хостами.

Для аналізу TCP-сеансів між двома хостами я спочатку виконала запис трафіку за допомогою tcpdump

```
(kali@kali)-[~/lab1bkm/task3]
$ sudo tcpdump -i eth1 -w task3_netcat.pcap
[sudo] password for kali:
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C47 packets captured
47 packets received by filter
0 packets dropped by kernel
```

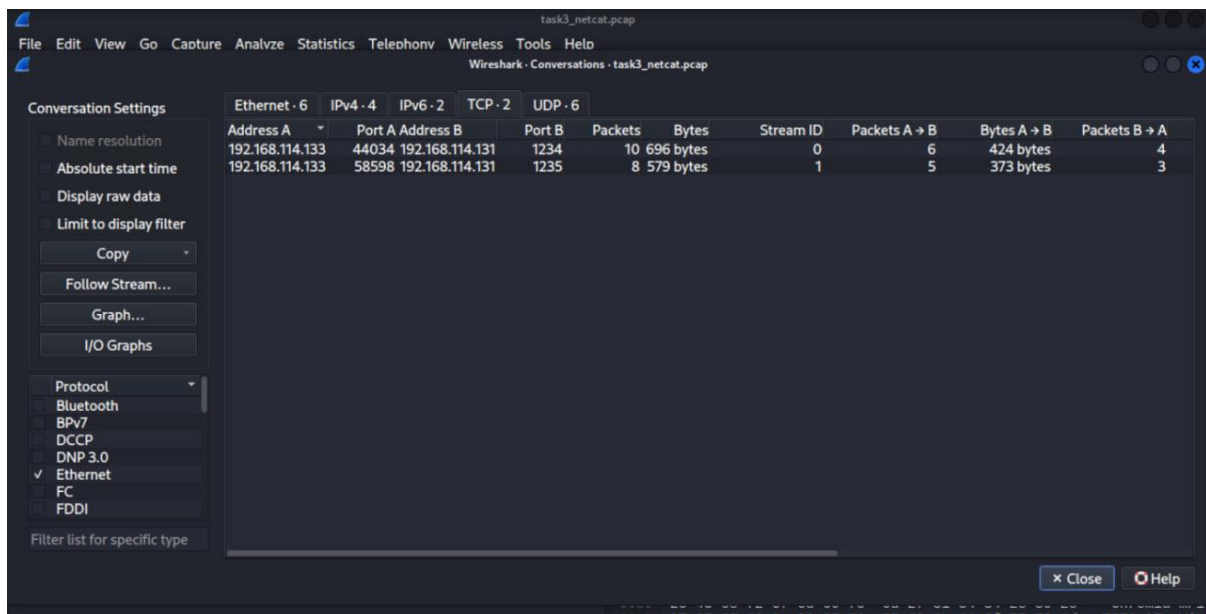
Після цього я повторила дії попереднього завдання, а саме встановила TCP-з'єднання через netcat, передала текстові повідомлення, передала файл

У результаті було записано мережевий трафік у файл task3_netcat.pcap

Далі відкриваю цей файл у Wireshark

Для того щоб визначити скільки TCP-сеансів, я використала Statistics -- Conversations - - TCP

У результаті я побачила, що між хостами було встановлено 2 TCP-сеанси

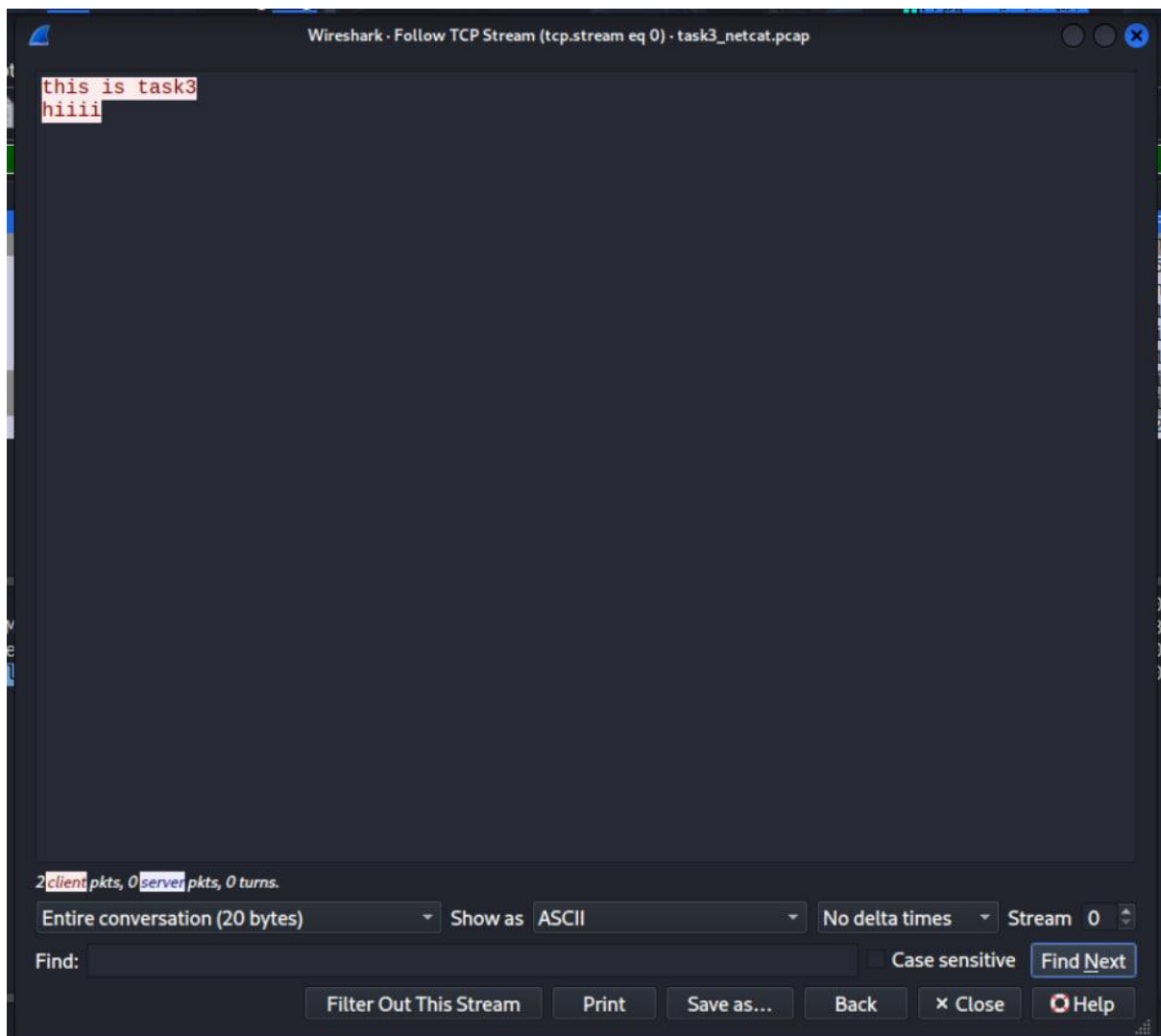


Які в цих сеансах передавались файли.

Відфільтрую лише tcp пакети з портом 1234

No.	Time	Source	Destination	Protocol	Length	Info
6	21.728809	192.168.114.133	192.168.114.131	TCP	74	44034 → 1234 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM 1
7	21.729564	192.168.114.131	192.168.114.133	TCP	74	1234 → 44034 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
8	21.730242	192.168.114.133	192.168.114.131	TCP	66	44034 → 1234 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=17743844
13	30.498317	192.168.114.133	192.168.114.131	TCP	80	44034 → 1234 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=14 TSval=17
14	30.498443	192.168.114.131	192.168.114.133	TCP	66	1234 → 44034 [ACK] Seq=1 Ack=15 Win=65152 Len=0 TSval=1936442
15	33.572069	192.168.114.133	192.168.114.131	TCP	72	44034 → 1234 [PSH, ACK] Seq=15 Ack=1 Win=64256 Len=6 TSval=17
16	33.572160	192.168.114.131	192.168.114.133	TCP	66	1234 → 44034 [ACK] Seq=1 Ack=21 Win=65152 Len=0 TSval=1936445
20	97.312138	192.168.114.133	192.168.114.131	TCP	66	44034 → 1234 [FIN, ACK] Seq=21 Ack=1 Win=64256 Len=0 TSval=17
21	97.312575	192.168.114.131	192.168.114.133	TCP	66	1234 → 44034 [FIN, ACK] Seq=1 Ack=22 Win=65152 Len=0 TSval=19
22	97.314395	192.168.114.133	192.168.114.131	TCP	66	44034 → 1234 [ACK] Seq=22 Ack=2 Win=64256 Len=0 TSval=1774460

У першому TCP-сеансі (порт 1234) передавався текст, який я вводила через netcat



У другому TCP-сеансі (порт 1235) передавався файл lab.txt

task3_netcat.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port == 1235

No.	Time	Source	Destination	Protocol	Length	Info
35	148.731719	192.168.114.133	192.168.114.131	TCP	74	58598 → 1235 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
36	148.732508	192.168.114.131	192.168.114.133	TCP	74	1235 → 58598 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
37	148.733392	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=17745034
38	148.734189	192.168.114.133	192.168.114.131	TCP	101	58598 → 1235 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=35 TSval=17
39	148.734196	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [FIN, ACK] Seq=36 Ack=1 Win=64256 Len=0 TSval=17
40	148.734229	192.168.114.131	192.168.114.133	TCP	66	1235 → 58598 [ACK] Seq=1 Ack=36 Win=65152 Len=0 TSval=1936552
41	148.734642	192.168.114.131	192.168.114.133	TCP	66	1235 → 58598 [FIN, ACK] Seq=1 Ack=37 Win=65152 Len=0 TSval=19
42	148.735073	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [ACK] Seq=37 Ack=2 Win=64256 Len=0 TSval=1774503

Wireshark · Follow TCP Stream (tcp.stream eq 1) · task3_netcat.pcap

i like potato and dont like tomato

1 client pkt, 0 server pkts, 0 turns.

Entire conversation (35 bytes) Show as ASCII No delta times Stream 1

Find: Case sensitive Find Next

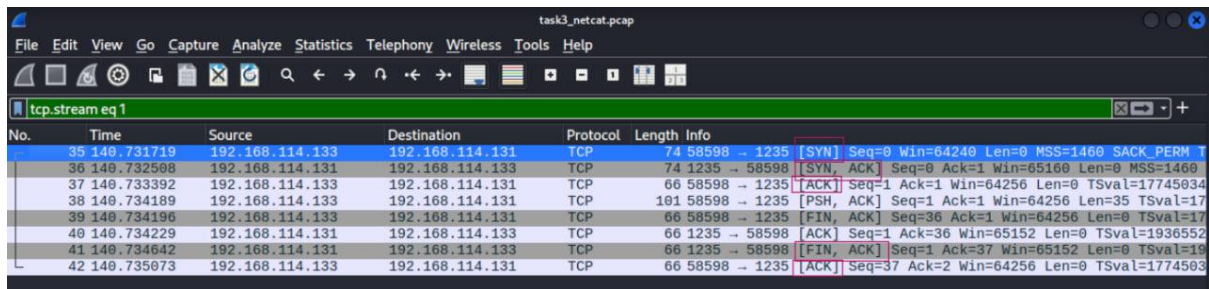
Filter Out This Stream Print Save as... Back × Close Help

Які коди повідомлень та які заголовки використовувались

Під час аналізу пакетів у Wireshark я побачила, що для встановлення TCP-з'єднання використовувались SYN; SYN, ACK; ACK

Під час передачі даних: PSH, ACK

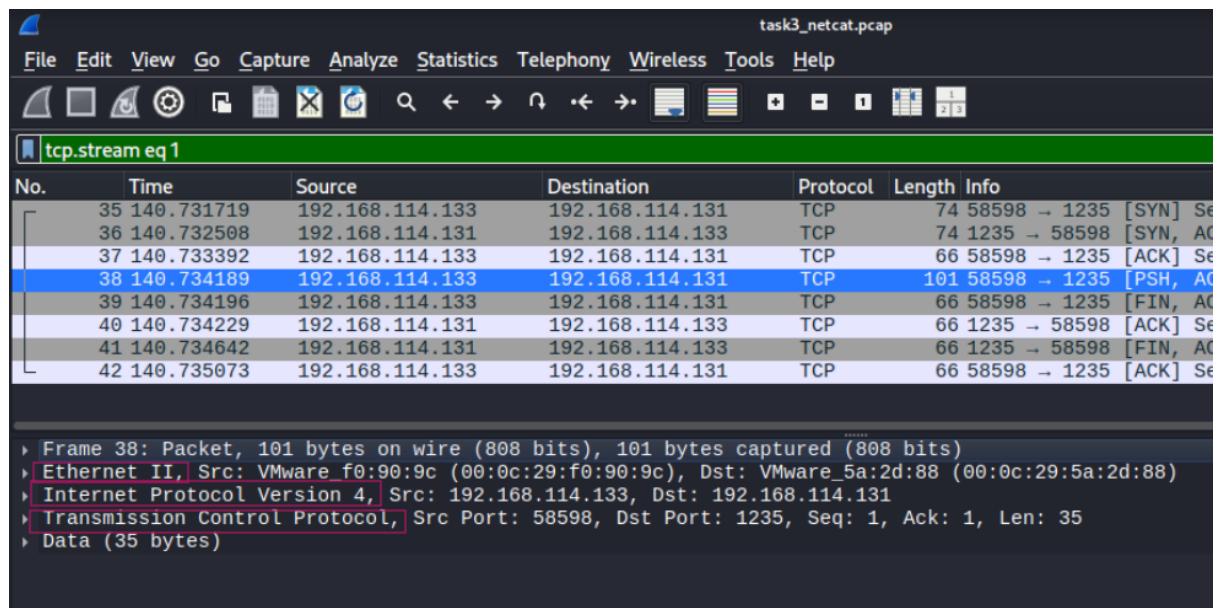
Для завершення з'єднання: FIN, ACK



The image shows a Wireshark packet capture titled 'task3_netcat.pcap'. The filter is 'tcp.stream eq 1'. The packet list shows a sequence of packets from 35 to 42. Packets 35, 36, 37, and 38 are SYN, ACK, and PSH/ACK respectively. Packets 39, 40, 41, and 42 are FIN, ACK, FIN, and ACK respectively, completing the connection termination. The packet details for packet 38 are expanded, showing the TCP header with Seq=1, Ack=1, Win=64256, Len=35, and TSval=17.

No.	Time	Source	Destination	Protocol	Length	Info
35	140.731719	192.168.114.133	192.168.114.131	TCP	74	58598 → 1235 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM T
36	140.732508	192.168.114.131	192.168.114.133	TCP	74	1235 → 58598 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
37	140.733392	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=17745034
38	140.734189	192.168.114.133	192.168.114.131	TCP	101	58598 → 1235 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=35 TSval=17
39	140.734196	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [FIN, ACK] Seq=36 Ack=1 Win=64256 Len=0 TSval=17
40	140.734229	192.168.114.131	192.168.114.133	TCP	66	1235 → 58598 [ACK] Seq=1 Ack=36 Win=65152 Len=0 TSval=1936552
41	140.734642	192.168.114.131	192.168.114.133	TCP	66	1235 → 58598 [FIN, ACK] Seq=1 Ack=37 Win=65152 Len=0 TSval=19
42	140.735073	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [ACK] Seq=37 Ack=2 Win=64256 Len=0 TSval=1774503

У кожному пакеті були такі заголовки:



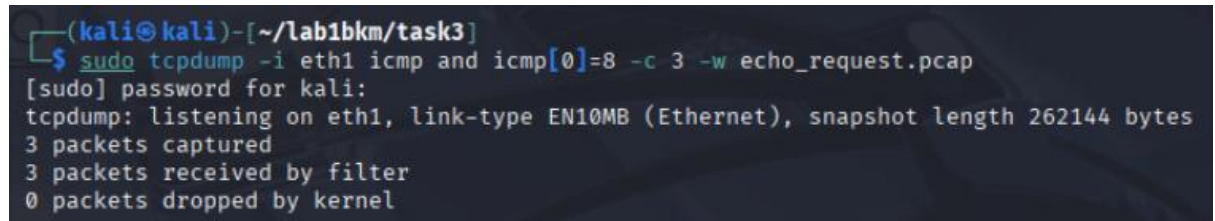
The image shows the packet details for packet 38 in Wireshark. The packet is a TCP ACK with sequence number 1, acknowledgment number 1, window size 64256, and length 35. The details pane shows the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header.

No.	Time	Source	Destination	Protocol	Length	Info
35	140.731719	192.168.114.133	192.168.114.131	TCP	74	58598 → 1235 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM T
36	140.732508	192.168.114.131	192.168.114.133	TCP	74	1235 → 58598 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
37	140.733392	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=17745034
38	140.734189	192.168.114.133	192.168.114.131	TCP	101	58598 → 1235 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=35 TSval=17
39	140.734196	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [FIN, ACK] Seq=36 Ack=1 Win=64256 Len=0 TSval=17
40	140.734229	192.168.114.131	192.168.114.133	TCP	66	1235 → 58598 [ACK] Seq=1 Ack=36 Win=65152 Len=0 TSval=1936552
41	140.734642	192.168.114.131	192.168.114.133	TCP	66	1235 → 58598 [FIN, ACK] Seq=1 Ack=37 Win=65152 Len=0 TSval=19
42	140.735073	192.168.114.133	192.168.114.131	TCP	66	58598 → 1235 [ACK] Seq=37 Ack=2 Win=64256 Len=0 TSval=1774503

Frame 38: Packet, 101 bytes on wire (808 bits), 101 bytes captured (808 bits)
Ethernet II, Src: VMware_f0:90:9c (00:0c:29:f0:90:9c), Dst: VMware_5a:2d:88 (00:0c:29:5a:2d:88)
Internet Protocol Version 4, Src: 192.168.114.133, Dst: 192.168.114.131
Transmission Control Protocol, Src Port: 58598, Dst Port: 1235, Seq: 1, Ack: 1, Len: 35
Data (35 bytes)

4. Відокремте окремими .pcap файлам пакети echo request (три послідовних пакети), TCP SYN, ARP request та ARP reply, що передавались між hostA та hostB

1. ICMP echo request (ping)

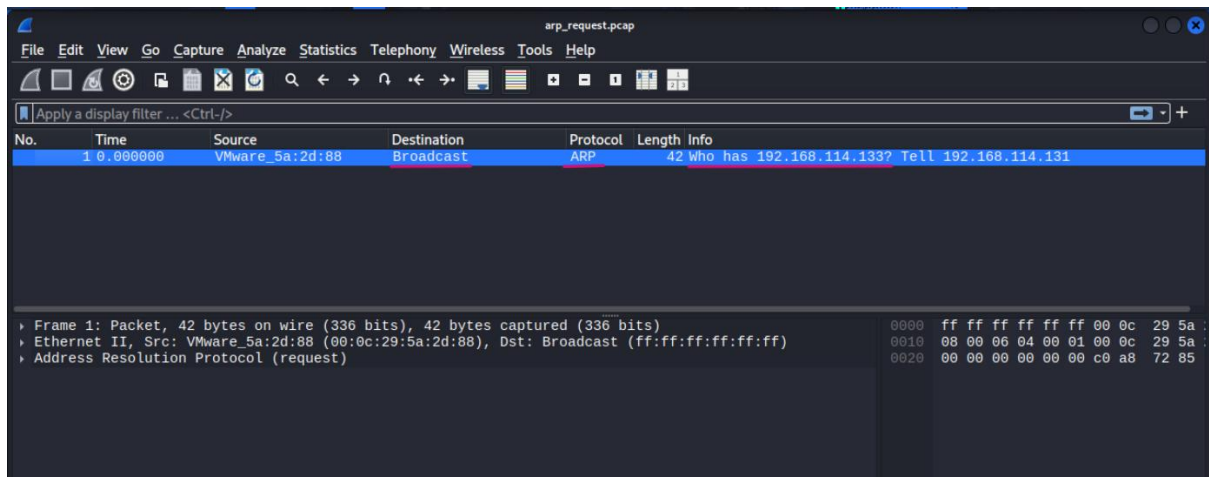


```
(kali@kali)-[~/lab1bkm/task3]
$ sudo tcpdump -i eth1 icmp and icmp[0]=8 -c 3 -w echo_request.pcap
[sudo] password for kali:
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
3 packets captured
3 packets received by filter
0 packets dropped by kernel
```



```
(kali㉿kali)-[~/lab1bkm/task3]
$ sudo arp -d 192.168.114.133

(kali㉿kali)-[~/lab1bkm/task3]
$ sudo tcpdump -i eth1 arp and arp[6:2]=1 -c 1 -w arp_request.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
1 packet captured
1 packet received by filter
0 packets dropped by kernel
```

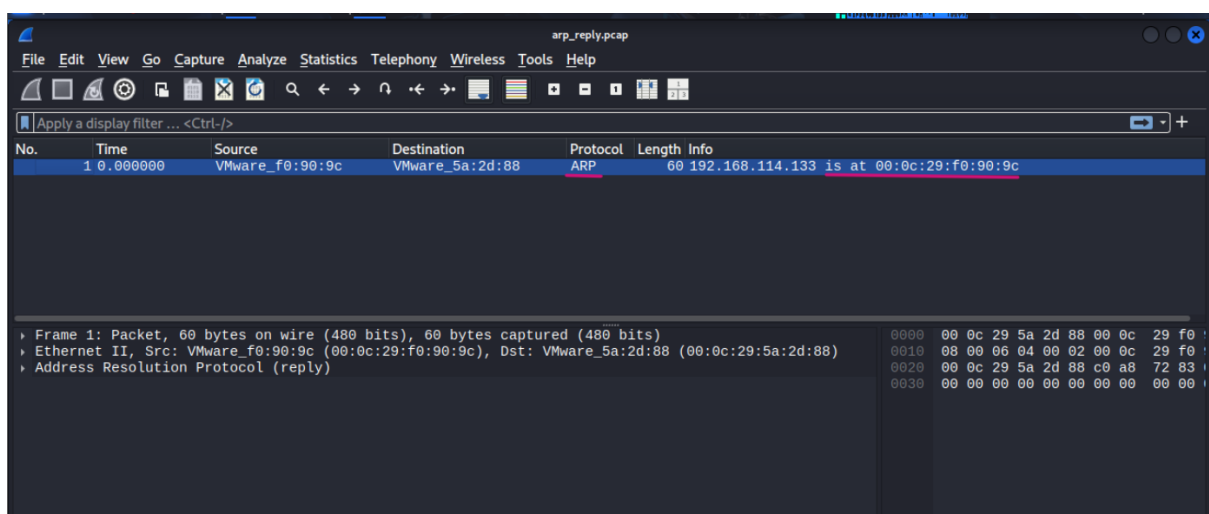


На скріні бачимо ARP request. Пакет має широкомовну адресу (Broadcast) і має запит типу "Who has", що означає пошук MAC-адреси за IP-адресою

4. ARP reply

```
(kali㉿kali)-[~/lab1bkm/task3]
$ sudo arp -d 192.168.114.133

(kali㉿kali)-[~/lab1bkm/task3]
$ sudo tcpdump -i eth1 arp and arp[6:2]=2 -c 1 -w arp_reply.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
1 packet captured
1 packet received by filter
0 packets dropped by kernel
```



На екрані бачимо ARP reply це відповідь на ARP-запит. Пакет має MAC-адресу відповідного хоста, це підтверджується "is at".

5. Відтворіть записані пакети ARP reply на hostC, послуговуючись

а) `trafgen -d <interface> -i <pktfile.txf> -c <packetcount>`

та

б) `tcpreplay`

Для використання .pcap файлів у `trafgen` пакети потрібно перетворити в .txf формат за допомогою `netsniff-ng`:

`netsniff-ng --in <file.pcap> --out <file.txf> -s`

Для копіювання файлів між хостами використовуйте `winscp` (`winscp.net`) або можливості `ssh`.

Попередньо скинувши ARP кеш на обох хостах, спостерігайте відповіді `hostA` на дані пакети та зміни у ARP таблиці (`arp -en` або `arp -a` для Windows)

*Також для відтворення пакетів можуть використовуватись `pktgen` або `scapy`

Перекидую `arp_reply.pcap` на **hostB** (калі2)

```
(kali@kali)-[~/lab1bkm/task3]
$ scp ~/lab1bkm/task3/arp_reply.pcap kali@192.168.114.133:/home/kali/lab1bkm
The authenticity of host '192.168.114.133 (192.168.114.133)' can't be established.
ED25519 key fingerprint is SHA256:X+XNccj86w7Z3phX6I8nDnkStoz30ymYajbQKVBg7g.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.114.133' (ED25519) to the list of known hosts.
kali@192.168.114.133's password:
arp_reply.pcap
100% 100 33.6KB/s 00:00
```

Конвертую на **hostB** (калі 2) у .txf

```
(kali@kali2)-[~/lab1bkm]
$ sudo netsniff-ng --in arp_reply.pcap --out arp_reply.txf -s
Running! Hang up with ^C!

1 packets outgoing
0 packets truncated in file
60 bytes outgoing
0 sec, 1625 usec in total

(kali@kali2)-[~/lab1bkm]
$ ls
arp_reply.pcap  arp_reply.txf  lab.txt
```

На Kali2 (hostB) змінюю MAC-адресу та IP-адресу мережевого інтерфейсу для імітації іншого вузла мережі (hostC). В результаті інтерфейс отримав нову MAC-адресу та IP-адресу 192.168.114.200

змінюю MAC на hostC (Kali2)

```
(kali@kali2)-[~/lab1bkm]
$ sudo ip link set dev eth1 down
[sudo] password for kali:

(kali@kali2)-[~/lab1bkm]
$ sudo ip link set dev eth1 address 02:11:22:33:44:55

(kali@kali2)-[~/lab1bkm]
$ sudo ip link set dev eth1 up

(kali@kali2)-[~/lab1bkm]
$ ip link show eth1
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 02:11:22:33:44:55 brd ff:ff:ff:ff:ff:ff permaddr 00:0c:29:f0:90:9c
```

змінюю IP на hostC (Kali2)

```
(kali@kali2)-[~/lab1bkm]
$ sudo ip link set eth1 down

(kali@kali2)-[~/lab1bkm]
$ sudo ip addr flush dev eth1

(kali@kali2)-[~/lab1bkm]
$ sudo ip addr add 192.168.114.200/24 dev eth1

(kali@kali2)-[~/lab1bkm]
$ sudo ip link set eth1 up

(kali@kali2)-[~/lab1bkm]
$ ip -br a
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        DOWN
eth1        UP        192.168.114.200/24
```

Зв'язок є

```
(kali@kali)-[~/lab1bkm/task3]
$ ping -c 2 192.168.114.200
PING 192.168.114.200 (192.168.114.200) 56(84) bytes of data.
64 bytes from 192.168.114.200: icmp_seq=1 ttl=64 time=16.1 ms
64 bytes from 192.168.114.200: icmp_seq=2 ttl=64 time=0.760 ms

— 192.168.114.200 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.760/8.453/16.146/7.693 ms
```

a) *trafgen -d -i -c <packetcount*

ARP таблиця до hostA (Kali1)

```
(kali@kali)-[~/lab1bkm/task3]
$ arp -en
Address          HWtype  HWaddress  Flags Mask  Iface
192.168.114.200   ether    02:11:22:33:44:55  C          eth1
192.168.114.254   ether    00:50:56:ed:ac:f4  C          eth1
192.168.114.134   ether    02:11:22:33:44:55  C          eth1
```

Роблю ping до IP-адреси 192.168.114.133. Відповідь не отримана, так як такого хоста в мережі не існує

```
(kali@kali)-[~/lab1bkm/task3]
$ ping -c 1 192.168.114.133
PING 192.168.114.133 (192.168.114.133) 56(84) bytes of data.

--- 192.168.114.133 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

на калі 2(hostC) я запустила trafgen і зупинила після відправки пакетів

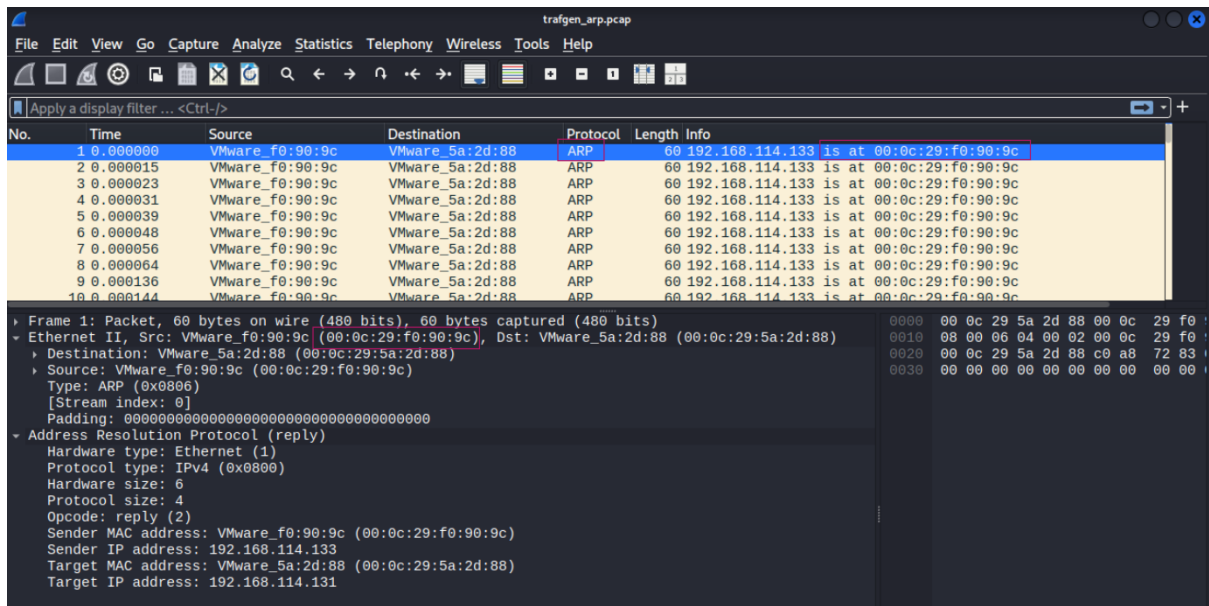
```
(kali@kali2)-[~/lab1bkm]
$ sudo trafgen -d eth1 -i arp_reply.txf
2 packets to schedule
120 bytes in total
Running! Hang up with ^C!

^C
228836 packets outgoing
13730160 bytes outgoing
6 sec, 279716 usec on CPU0 (11777 packets)
6 sec, 270468 usec on CPU1 (217059 packets)
```

Після відтворення пакету в ARP таблиці з'явився запис для IP-адреси 192.168.114.133, хоча даний хост фактично відсутній у мережі

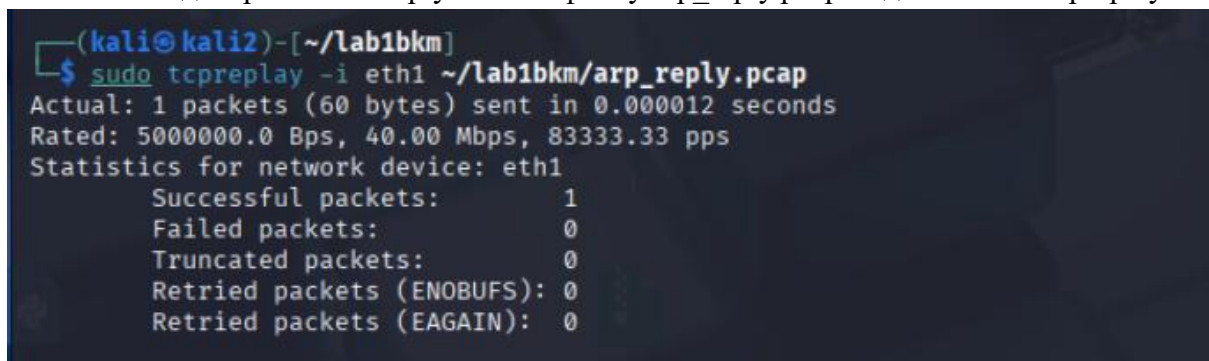
```
(kali@kali)-[~/lab1bkm/task3]
$ arp -en
Address          HWtype  HWaddress  Flags Mask  Iface
192.168.114.133   ether    00:0c:29:f0:90:9c  C          eth1
192.168.114.200   ether    02:11:22:33:44:55  C          eth1
192.168.114.254   ether    00:50:56:ed:ac:f4  C          eth1
192.168.114.134   ether    02:11:22:33:44:55  C          eth1
```

У Wireshark у файлі trafgen_arp.pcap видно ARP reply 192.168.114.133 is at... тобто пакет дійшов до hostA (Kali1), Це означає що ARP-відповідь була успішно передана мережею.

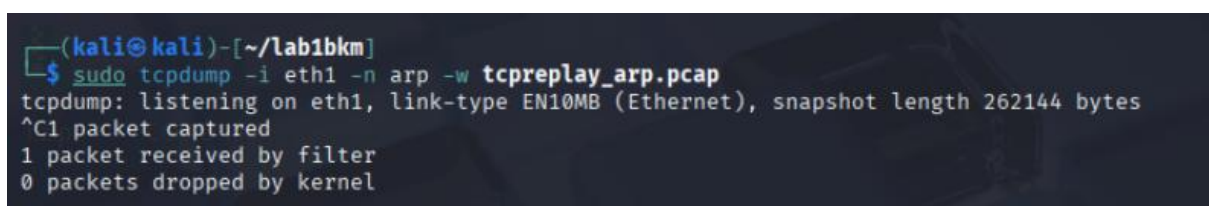


б) tcpreplay

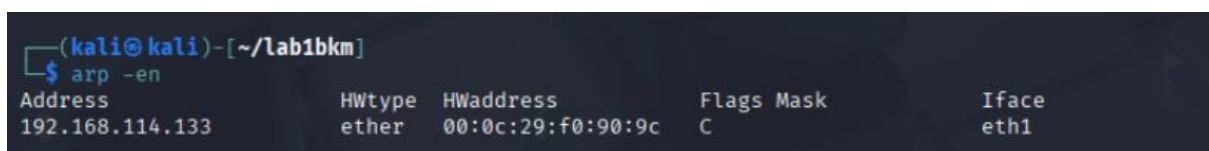
На hostC я відтворила ARP reply пакет з файлу arp_reply.pcap за допомогою tcpreplay



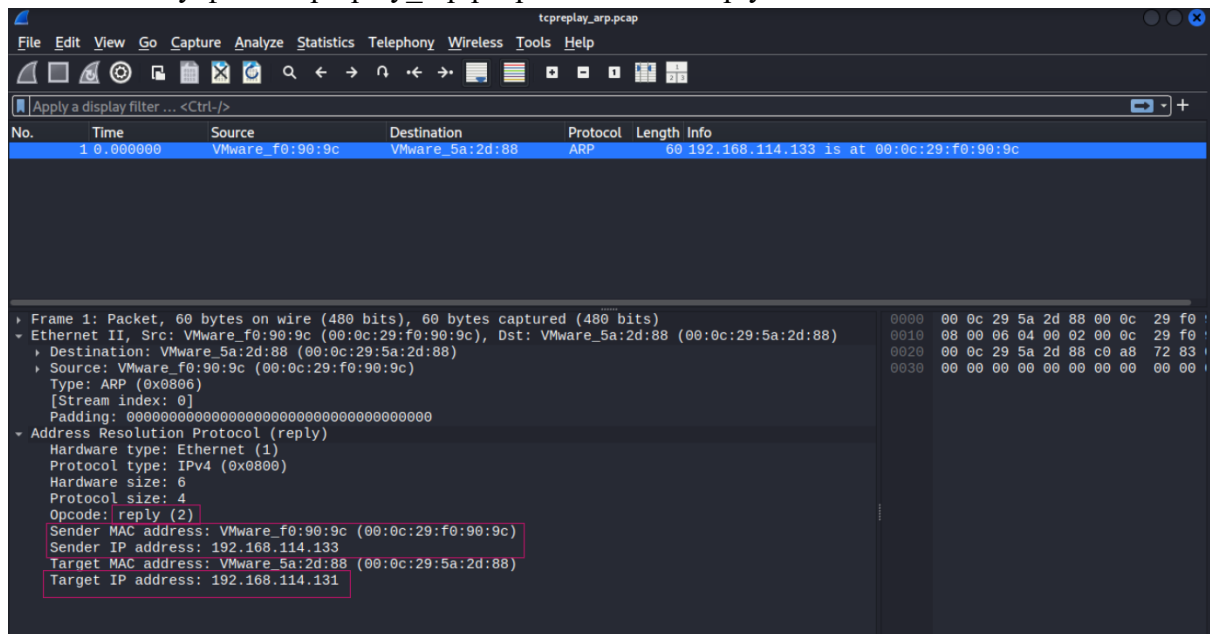
За допомогою tcpdump бачу що отримали ARP-пакет



Після в ARP-таблиці з'явився запис для IP-адреси 192.168.114.133



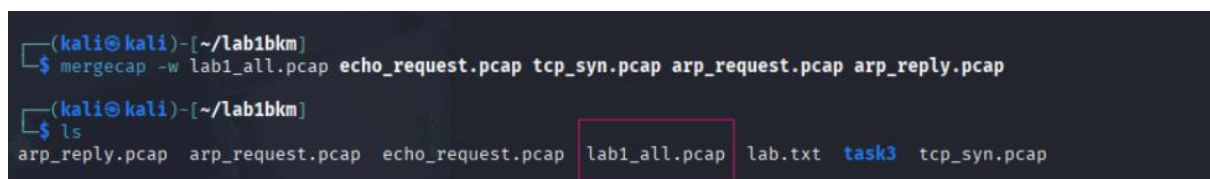
У Wireshark у файлі tcpreplay_arp.pcap видно ARP reply 192.168.114.133 is at ...



це ARP Reply (opcode = 2), у якому хост з IP-адресою 192.168.114.133 повідомляє свою MAC-адресу для хоста 192.168.114.131, тобто можна ствердити, що після tcpreplay хост отримав ARP-відповідь і оновив ARP-кеш

6. За допомогою інформації Wireshark зобразіть ієрархічно (у вигляді стеку протоколів) структуру протоколів, що передавались у мережі. Які заголовки мають відповідні пакети? Позначте це також на вашому малюнку. Які MAC та IP адреси мають хости мережі. (Зберіть дану статистику, використовуючи можливості wireshark). Як відокремити пакети кожного з протоколів, що вказано вище?

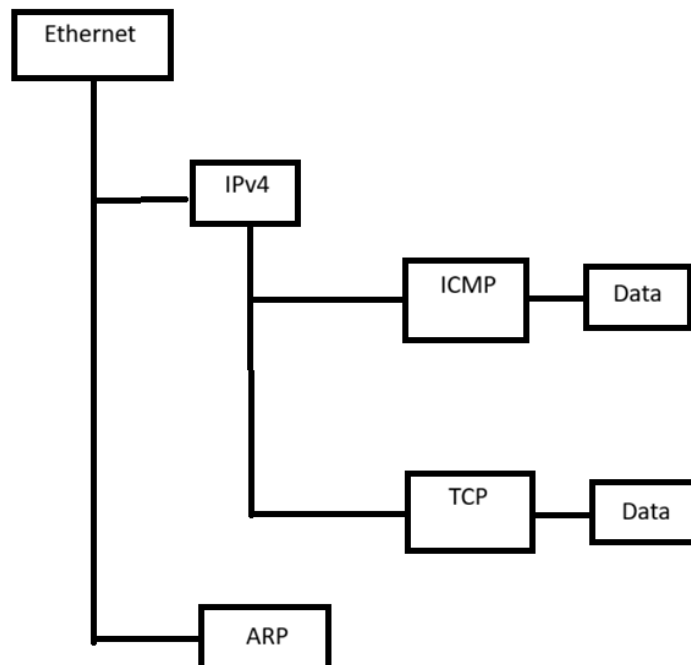
Зроблю з усіх .pcap один файл



Перейду у wireshark та перегляну Statistics -- Protocol Hierarchy

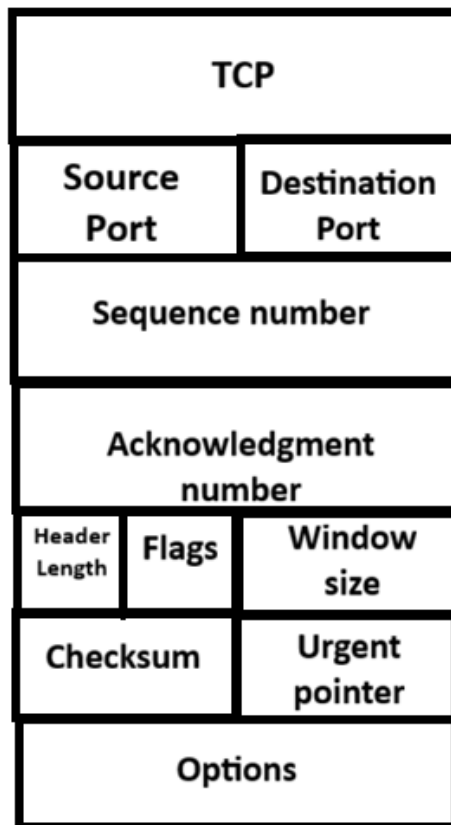
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	6	100.0	470	30	0	0	0	6
Ethernet	100.0	6	21.7	102	6	0	0	0	6
Internet Protocol Version 4	66.7	4	17.0	80	5	0	0	0	4
Transmission Control Protocol	16.7	1	8.5	40	2	1	40	2	1
Internet Control Message Protocol	50.0	3	40.9	192	12	3	192	12	3
Address Resolution Protocol	33.3	2	11.9	56	3	2	56	3	2

На основі аналізу пакетів у Wireshark я побудувала ієрархічну структуру протоколів (стек протоколів). Передача даних відбувається через вкладеність протоколів



Які заголовки мають відповідні пакети? Позначте це також на вашому малюнку

TCP



```

> Ethernet II, Src: VMware_f0:90:9c (00:0c:29:f0:90:9c), Dst: VMware_5a:2d:8 0000
> Internet Protocol Version 4, Src: 192.168.114.133, Dst: 192.168.114.131 0010
> Transmission Control Protocol, Src Port: 36246, Dst Port: 1234, Seq: 0, Le 0020
  Source Port: 36246 0030
  Destination Port: 1234 0040
  [Stream index: 0]
  [Stream Packet Number: 1]
  [Conversation completeness: Incomplete, SYN_SENT (1)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 2271135185
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1010 .... = Header Length: 40 bytes (10)
  Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0x0f1e [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, I
  [Timestamps]
  [Client Contiguous Streams: 0]
  [Server Contiguous Streams: 1]

```

ICMP

ICMP		
Type	Code	Checksum
Identifier		Sequence Number
ICMP Data		

```

Internet Control Message Protocol
  Type: Echo (ping) request (8)
  Code: 0
  Checksum: 0xe0ea [correct]
  [Checksum Status: Good]
  Identifier (BE): 2 (0x0002)
  Identifier (LE): 512 (0x0200)
  Sequence Number (BE): 1 (0x0001)
  Sequence Number (LE): 256 (0x0100)
  [No response seen]
  ICMP Data: 9fd9986900000000000008fc08000000000010111213141

```

ARP

ARP		
Hardware type		Protocol type
Hardware size	Protocol size	Opcode
Sender MAC Address		
Sender IP Address		
Target MAC Address		
Target IP Address		

```

Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: VMware_f0:90:9c (00:0c:29:f0:90:9c)
  Sender IP address: 192.168.114.133
  Target MAC address: VMware_5a:2d:88 (00:0c:29:5a:2d:88)
  Target IP address: 192.168.114.131

```

Які MAC та IP адреси мають хости мережі. (Зберіть дану статистику, використовуючи можливості wireshark)

Захожу в Wireshark Statistics -- Endpoints

MAC адреси

Ethernet · 3	IPv4 · 2	IPv6	TCP · 2	UDP			
Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	
00:0c:29:5a:2d:88	6	470 bytes	1	42 bytes	5	428 bytes	
00:0c:29:f0:90:9c	5	428 bytes	5	428 bytes	0	0 bytes	
ff:ff:ff:ff:ff:ff	1	42 bytes	0	0 bytes	1	42 bytes	

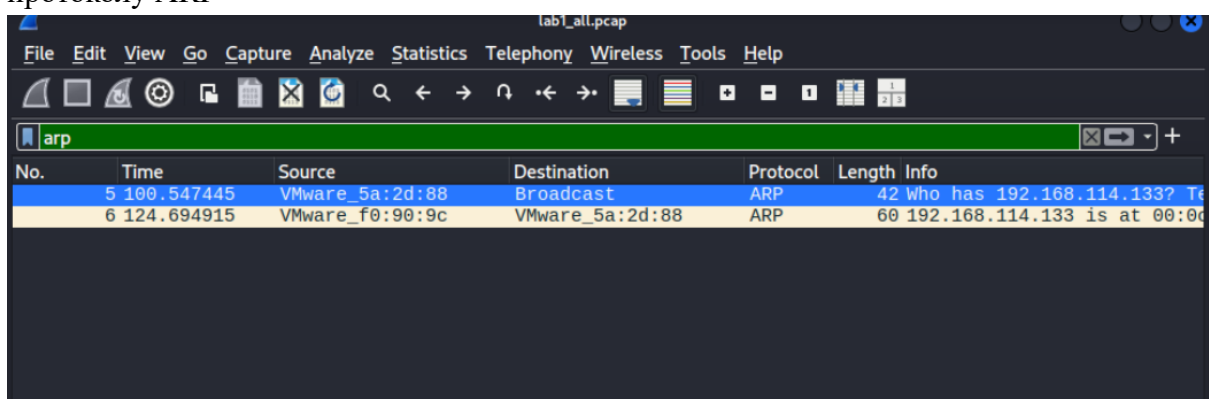
IP адреси

Ethernet · 3	IPv4 · 2	IPv6	TCP · 2	UDP								
Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Country	City	Latitude	Longitude	AS N	
192.168.114.131	4	368 bytes	0	0 bytes	4	368 bytes						
192.168.114.133	4	368 bytes	4	368 bytes	0	0 bytes						

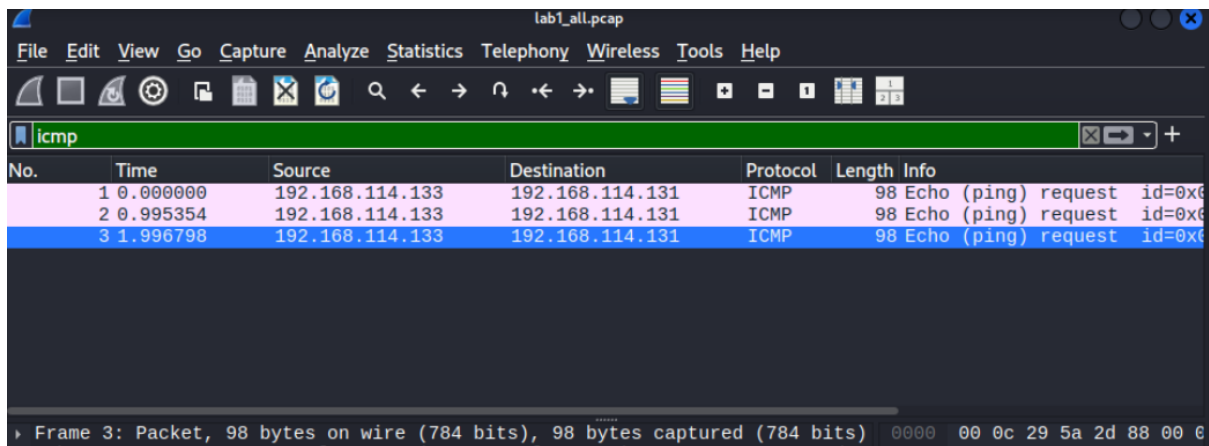
Як відокремити пакети кожного з протоколів, що вказано вище?

Для цього у Display Filter пишу протокол для шуканих пакетів

Для відокремлення ARP-пакетів використаю фільтр який показує лише пакети протоколу ARP



Для відокремлення ICMP-пакетів аналогічно використаю фільтр icmp



The image shows a Wireshark interface with a packet capture named 'lab1_all.pcap'. The filter is set to 'icmp'. The packet list shows three ICMP Echo (ping) requests. The packet details pane shows the structure of the first packet: Ethernet II, Internet Protocol Version 4, and ICMP Echo (ping) request.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.114.133	192.168.114.131	ICMP	98	Echo (ping) request id=0x0
2	0.995354	192.168.114.133	192.168.114.131	ICMP	98	Echo (ping) request id=0x0
3	1.996798	192.168.114.133	192.168.114.131	ICMP	98	Echo (ping) request id=0x0

Frame 3: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)

7. За допомогою *iperf*, оцініть ефективну ПС при використанні протоколу TCP між деякими двома хостами в мережі:

***iperf -s* //на сервері**

***iperf -c* //на клієнті**

Змінюючи параметр тривалості з'єднання (*iperf -c -t*), наприклад, зі значеннями 5,10 та 30 секунд та кількості TCP-потоків (1,2,3,4) *iperf -c -P* зробіть висновок щодо залежності ефективності TCP-з'єднання відносно даних параметрів.

Порівняйте ПС, яку дає протокол TCP, з ПС протоколу UDP: *iperf -c -u -b*

Я запустила сервер *iperf* на калі1 (hostA) та клієнт на калі2(hostB)



```
(kali@kali)-[~]
$ iperf -s

Server listening on TCP port 5001
TCP window size: 128 KByte (default)

[ 1] local 192.168.114.131 port 5001 connected with 192.168.114.134 port 5542
[ ID] Interval      Transfer    Bandwidth
```

Я виміряла пропускну здатність TCP при 5 сек та 1,2,3 та 4 потоках

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 5 -P 4

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.114.134 port 37584 connected with 192.168.114.131 port 5001
[ 3] local 192.168.114.134 port 37602 connected with 192.168.114.131 port 5001
[ 4] local 192.168.114.134 port 37582 connected with 192.168.114.131 port 5001
[ 2] local 192.168.114.134 port 37604 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-5.0274 sec  468 MBytes   780 Mbits/sec
[ 2] 0.0000-5.0435 sec  371 MBytes   616 Mbits/sec
[ 4] 0.0000-5.0462 sec  416 MBytes   691 Mbits/sec
[ 3] 0.0000-5.0432 sec  540 MBytes   898 Mbits/sec
[SUM] 0.0000-5.0436 sec  1.75 GBytes  2.98 Gbits/sec
```

10 сек

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 10 -P 1

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.114.134 port 49992 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-10.0277 sec  3.48 GBytes   2.98 Gbits/sec

(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 10 -P 2

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 2] local 192.168.114.134 port 50354 connected with 192.168.114.131 port 5001
[ 1] local 192.168.114.134 port 50352 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-10.0158 sec  1.99 GBytes   1.70 Gbits/sec
[ 2] 0.0000-10.0382 sec  1.50 GBytes   1.28 Gbits/sec
[SUM] 0.0000-10.0382 sec  3.48 GBytes   2.98 Gbits/sec

(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 10 -P 3

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 3] local 192.168.114.134 port 57282 connected with 192.168.114.131 port 5001
[ 1] local 192.168.114.134 port 57284 connected with 192.168.114.131 port 5001
[ 2] local 192.168.114.134 port 57294 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-10.0325 sec  1.05 GBytes   895 Mbits/sec
[ 2] 0.0000-10.0329 sec  1.23 GBytes   1.05 Gbits/sec
[ 3] 0.0000-10.0330 sec  1.24 GBytes   1.07 Gbits/sec
[SUM] 0.0000-10.0330 sec  3.52 GBytes   3.01 Gbits/sec
```

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 10 -P 4
```

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

```
[ 2] local 192.168.114.134 port 36702 connected with 192.168.114.131 port 5001
[ 3] local 192.168.114.134 port 36722 connected with 192.168.114.131 port 5001
[ 4] local 192.168.114.134 port 36704 connected with 192.168.114.131 port 5001
[ 1] local 192.168.114.134 port 36710 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-10.0227 sec  841 MBytes   704 Mbits/sec
[ 3] 0.0000-10.0390 sec  839 MBytes   701 Mbits/sec
[ 2] 0.0000-10.0409 sec  822 MBytes   687 Mbits/sec
[ 4] 0.0000-10.0374 sec  636 MBytes   532 Mbits/sec
[SUM] 0.0000-10.0392 sec  3.06 GBytes  2.62 Gbits/sec
```

30 сек

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 30 -P 1
```

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

```
[ 1] local 192.168.114.134 port 36548 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0204 sec  10.8 GBytes  3.10 Gbits/sec
```

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 30 -P 2
```

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

```
[ 1] local 192.168.114.134 port 38156 connected with 192.168.114.131 port 5001
[ 2] local 192.168.114.134 port 38154 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0104 sec  4.10 GBytes  1.17 Gbits/sec
[ 2] 0.0000-30.0266 sec  4.36 GBytes  1.25 Gbits/sec
[SUM] 0.0000-30.0266 sec  8.46 GBytes  2.42 Gbits/sec
```

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 30 -P 3
```

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

```
[ 2] local 192.168.114.134 port 58742 connected with 192.168.114.131 port 5001
[ 1] local 192.168.114.134 port 58760 connected with 192.168.114.131 port 5001
[ 3] local 192.168.114.134 port 58740 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3] 0.0000-30.0274 sec  3.84 GBytes  1.10 Gbits/sec
[ 2] 0.0000-30.0449 sec  3.34 GBytes  955 Mbits/sec
[ 1] 0.0000-30.0460 sec  3.43 GBytes  980 Mbits/sec
[SUM] 0.0000-30.0460 sec  10.6 GBytes  3.03 Gbits/sec
```

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -t 30 -P 4

Client connecting to 192.168.114.131, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 2] local 192.168.114.134 port 44984 connected with 192.168.114.131 port 5001
[ 1] local 192.168.114.134 port 45012 connected with 192.168.114.131 port 5001
[ 3] local 192.168.114.134 port 44986 connected with 192.168.114.131 port 5001
[ 4] local 192.168.114.134 port 45014 connected with 192.168.114.131 port 5001
[ ID] Interval           Transfer     Bandwidth
[ 1] 0.0000-30.0725 sec  2.43 GBytes  693 Mbits/sec
[ 2] 0.0000-30.0724 sec  2.35 GBytes  670 Mbits/sec
[ 4] 0.0000-30.0712 sec  2.59 GBytes  740 Mbits/sec
[ 3] 0.0000-30.0720 sec  2.51 GBytes  717 Mbits/sec
[SUM] 0.0000-30.0724 sec  9.87 GBytes  2.82 Gbits/sec
```

Таблиця пропускної здатності у Gbit/s

	1 потік	2 потоки	3 потоки	4 потоки
5 сек	2.98	3.04	2.45	2.98
10 сек	2.98	2.98	3.01	2.62
30 сек	3.10	2.42	3.03	2.82

Найбільшу пропускну здатність можна відслідкувати при 1 потоці і максимальному часі, а взагалі зі збільшенням часу з'єднання пропускну здатність стає більш стабільною. Найкращі результати спостерігаються при 1-3 потоках. При використанні 4 потоків ефективність з'єднання трохи знижується і це може бути пов'язано з перевантаженням каналу або збільшенням кількості повторних передач, тож більше потоків не завжди краще.

Встановлюю ліміт передачі у 1 та 5 ГБ для того щоб побачити всю картину

```
(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -u -b 1G -t 10

Client connecting to 192.168.114.131, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.114.134 port 49697 connected with 192.168.114.131 port 5001
[ ID] Interval           Transfer     Bandwidth
[ 1] 0.0000-10.0001 sec  320 MBytes  269 Mbits/sec
[ 1] Sent 228545 datagrams
[ 1] Server Report:
[ ID] Interval           Transfer     Bandwidth           Jitter    Lost/Total Datagrams
[ 1] 0.0000-9.9982 sec   305 MBytes  256 Mbits/sec       0.000 ms 11252/228544 (0%)

(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -u -b 1G -t 30

Client connecting to 192.168.114.131, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.114.134 port 51409 connected with 192.168.114.131 port 5001
[ ID] Interval           Transfer     Bandwidth
[ 1] 0.0000-30.0001 sec  951 MBytes  266 Mbits/sec
[ 1] Sent 678063 datagrams
[ 1] Server Report:
[ ID] Interval           Transfer     Bandwidth           Jitter    Lost/Total Datagrams
[ 1] 0.0000-30.0024 sec   944 MBytes  264 Mbits/sec       0.000 ms 4472/678062 (0%)
```

```

(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -u -b 5G -t 30

Client connecting to 192.168.114.131, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.114.134 port 46785 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0001 sec  1.03 GBytes  295 Mbits/sec
[ 1] Sent 753259 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-29.9996 sec  985 MBytes   275 Mbits/sec   0.000 ms 50639/753258 (0%)

(kali@kali2)-[~]
$ iperf -c 192.168.114.131 -u -b 5G -t 10

Client connecting to 192.168.114.131, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.114.134 port 46359 connected with 192.168.114.131 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-10.0001 sec  319 MBytes   268 Mbits/sec
[ 1] Sent 227481 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-10.0012 sec  316 MBytes   265 Mbits/sec   0.000 ms 2142/227480 (0%)

```

Проаналізуючи результати, я можу сказати що швидкість UDP становить приблизно 260–295 Мбіт/с, при цьому маємо втрати пакетів та затримки, у TCP швидкість була вищою приблизно 2.5–3.1 Гбіт/с і передача без втрат.

TCP працює стабільніше і дає більшу швидкість, тому що контролює передачу даних, UDP не перевіряє доставку пакетів, через це частина даних губиться і швидкість виходить нижча, навіть якщо задавати велике значення -b

Висновки:

У ході лабораторної роботи я перевірила передачу мережного трафіка між двома хостами у віртуальній мережі. За допомогою tcpdump та Wireshark я перехопила і проаналізувала пакети ICMP, ARP та TCP і зберегла їх у .pcap файли. Я перевірила взаємну досяжність хостів, встановила TCP-з'єднання через netcat та передала текстові повідомлення і файл. Також я проаналізувала TCP-сеанси, мережні служби та оцінила пропускну здатність мережі за допомогою iperf. У результаті роботи я отримала практичні навички перехоплення та аналізу мережного трафіка